Von Neumann Architecture

Von Neumann Architecture

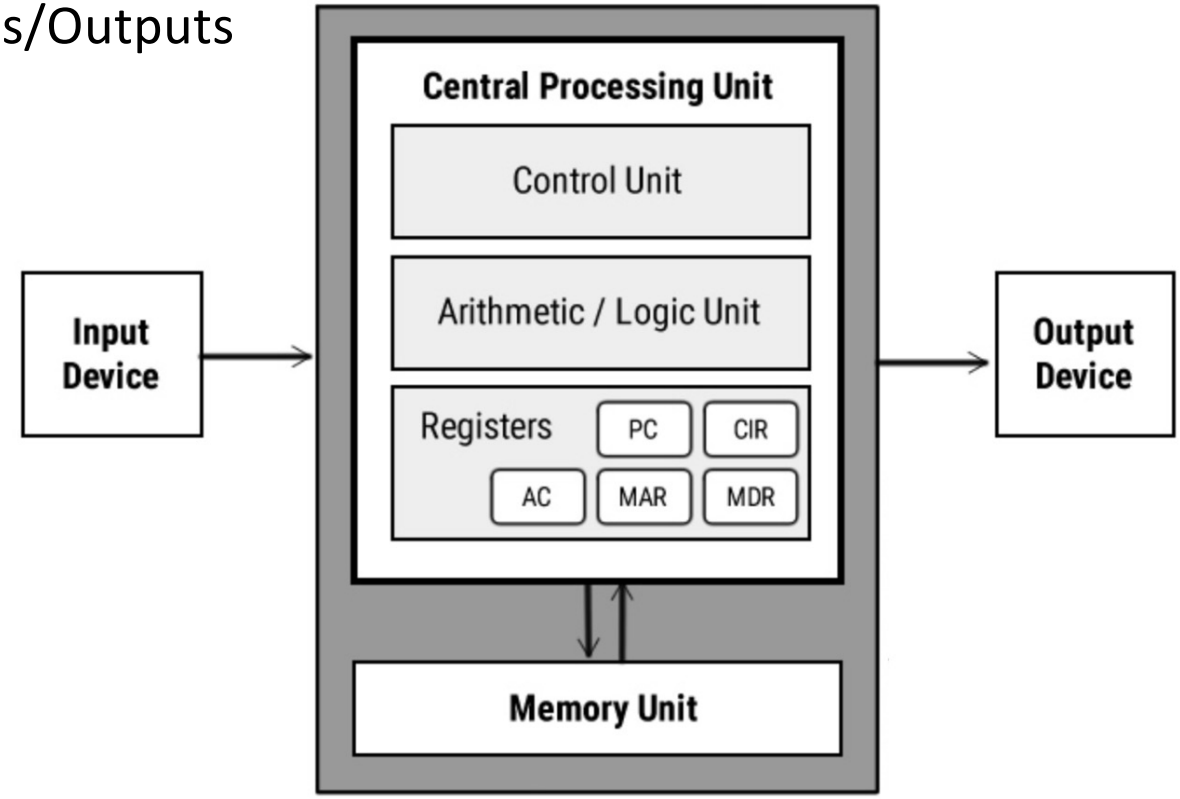
- Published by John von Neumann in 1945

- Stored Program Computers – Von Neumann architecture is based on the stored-program computer concept, where instruction data and program data are stored in the same memory.

- Fixed Program Computers – Their function is very specific, and they couldn’t be reprogrammed, e.g. Calculators.

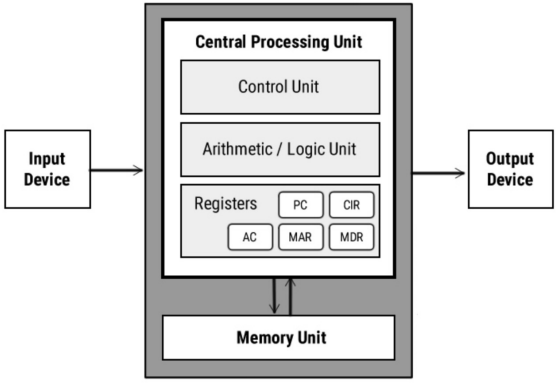
Von Neumann Architecture

- Computer architecture design consists of
  - Control Unit
  - Arithmetic and Logic Unit (ALU)
  - Registers
  - Memory Unit
  - Inputs/Outputs



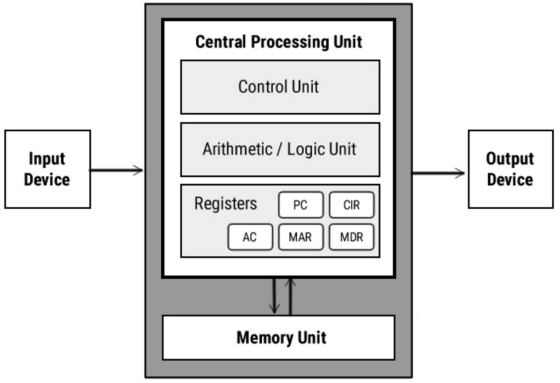
Von Neumann Architecture

- Central Processing Unit (CPU)
  - CPU is the electronic circuit responsible for executing the instructions of a computer program
  - Sometimes referred to as the microprocessor or processor
  - CPU contains the ALU, CU, and a variety of registers



Von Neumann Architecture

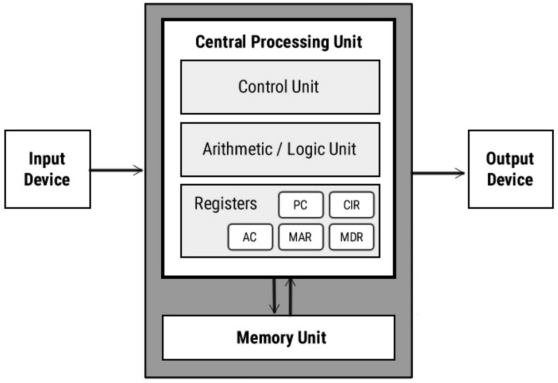
- Control Unit (CU)
  - CU controls the operation of the computer's ALU, memory, and input/output devices, telling them how to respond to the program instructions it has just read and interpreted from the memory unit.
  - CU also provides the timing and control signals required by other computer components.
- Arithmetic and Logic Unit (ALU)
  - ALU allows arithmetic (add, subtract, etc.) and logic (AND, OR, NOT etc.) operations to be carried out.



Von Neumann Architecture

- Registers
  - Registers are high speed storage areas in CPU. All data must be stored in a register before it can be processed.

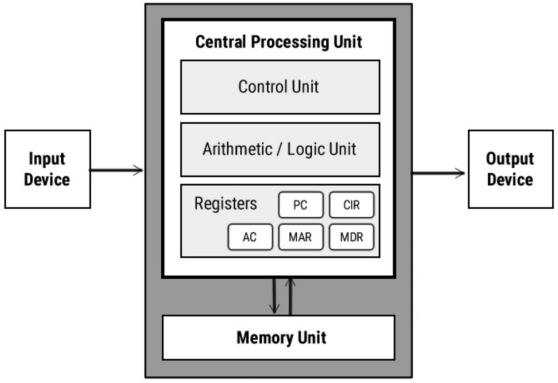
MAR	Memory Address Register	Holds the memory location of data that needs to be accessed
MDR	Memory Data Register	Holds data that is being transferred to or from memory
AC	Accumulator	Where intermediate arithmetic and logic results are stored
PC	Program Counter	Contains the address of the next instruction to be executed
CIR	Current Instruction Register	Contains the current instruction during processing



Von Neumann Architecture

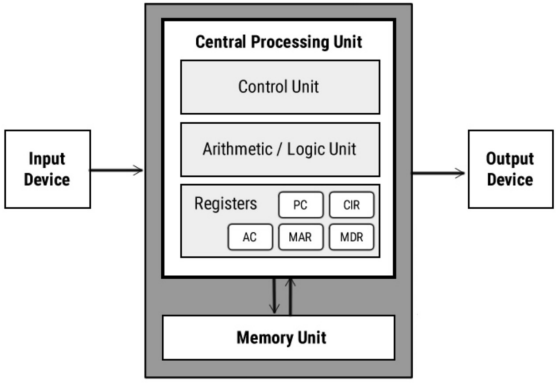
- Buses
  - Buses are the means by which data is transmitted from one part of a computer to another, connecting all major internal components to the CPU and memory.
  - A standard CPU system bus is comprised of a control bus, data bus and address bus.

Address Bus	Carries the addresses of data (but not the data) between the processor and memory
Data Bus	Carries data between the processor, the memory unit and the input/output devices
Control Bus	Carries control signals/commands from the CPU (and status signals from other devices) in order to control and coordinate all the activities within the computer



Von Neumann Architecture

- Memory Unit
  - The memory unit consists of RAM (primary or main memory). Unlike a hard drive (secondary memory), this memory is fast and directly accessible by the CPU.
  - RAM is split into partitions. Each partition consists of an address and its contents (both in binary form). The address will uniquely identify every location in the memory.
  - Loading data from permanent memory (hard drive), into the faster and directly accessible temporary memory (RAM), allows the CPU to operate much quicker.



Amdahl's Law

- Architecture design is very bottleneck-driven – make the common case fast, do not waste resources on a component that has little impact on overall performance/power
- Amdahl's Law: performance improvements through an enhancement is limited by the fraction of time the enhancement comes into play
- Example: a web server spends 40% of time in the CPU and 60% of time doing I/O – a new processor that is ten times faster results in a 36% reduction in execution time (speedup of 1.56) – Amdahl's Law states that maximum execution time reduction is 40% (max speedup of 1.66)

Common Principles

- Amdahl's Law
- Energy: systems leak energy even when idle
- Energy: performance improvements typically also result in energy improvements – less leakage
- 90-10 rule: 10% of the program accounts for 90% of execution time
- Principle of locality: the same data/code will be used again (temporal locality), nearby data/code will be touched next (spatial locality)

Single-/Multi-Cycle Design
Basic Pipelining

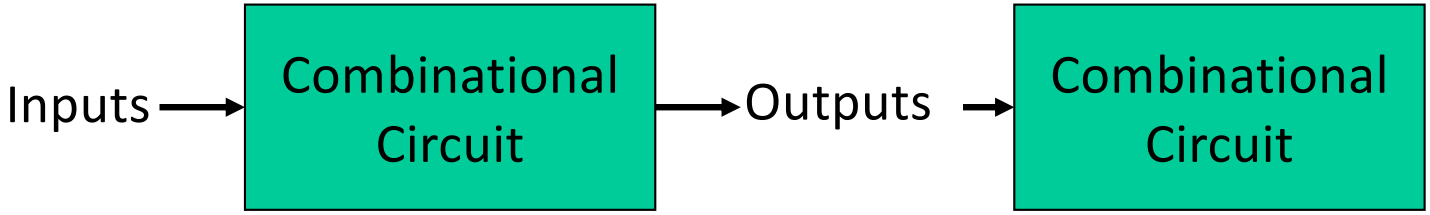
Clocks

- A microprocessor is composed of many different circuits that are operating simultaneously – if each circuit X takes in inputs at time TI_X, takes time TE_X to execute the logic, and produces outputs at time TO_X, imagine the complications in co-ordinating the tasks of every circuit

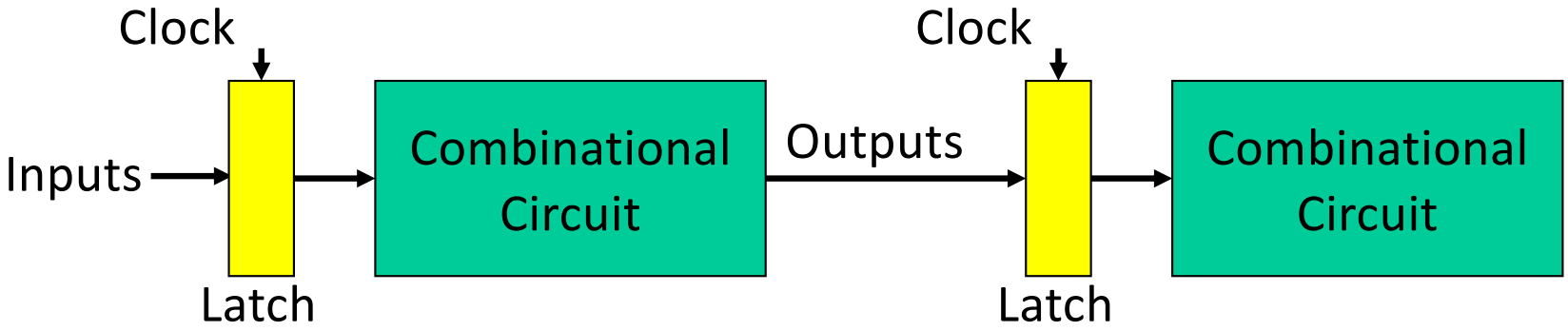
- A major school of thought (used in most processors built today): all circuits on the chip share a clock signal (a square wave) that tells every circuit when to accept inputs, how much time they have to execute the logic, and when they must produce outputs

Sequential Circuits

- Until now, circuits were combinational – when inputs change, the outputs change after a while (time = logic delay thru circuit)



- We want the clock to act like a start and stop signal – a "latch" is a storage device that separates these circuits – it ensures that the inputs to the circuit do not change during a clock cycle



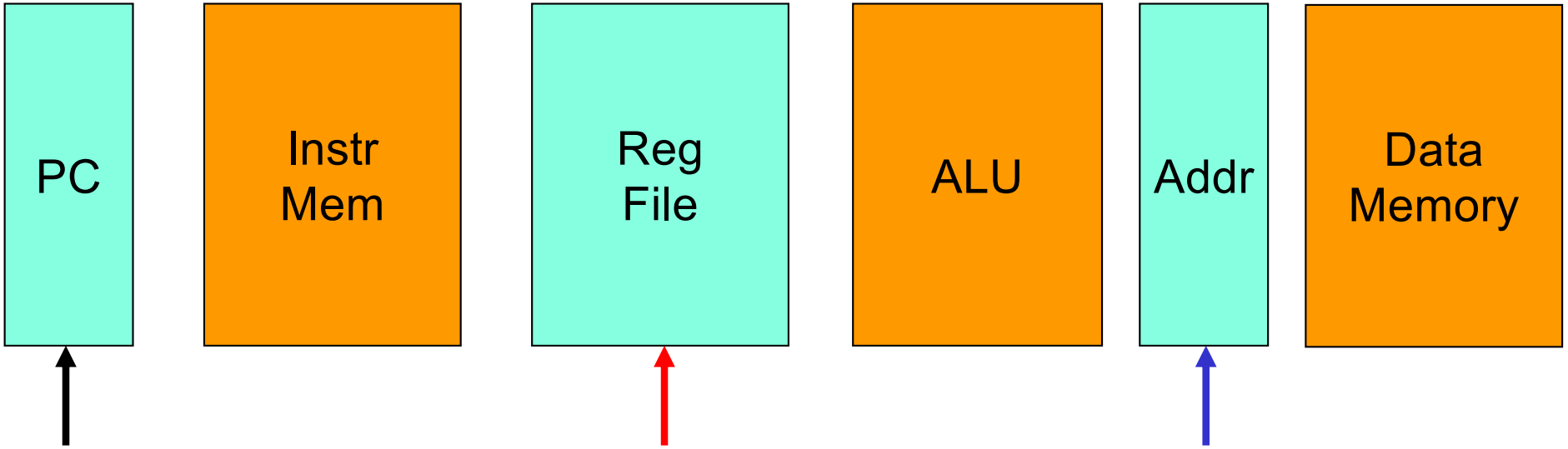
Basic MIPS Architecture

- Now that we understand clocks and storage of states, we'll design a simple CPU that executes:
  - basic math (add, sub, and, or, slt)
  - memory access (lw and sw)
  - branch and jump instructions (beq and j)

Implementation Overview

- We need memory
  - to store instructions
  - to store data
  - for now, let's make them separate units
- We need registers, ALU, and a whole lot of control logic
- CPU operations common to all instructions:
  - use the program counter (PC) to pull instruction out of instruction memory
  - read register values

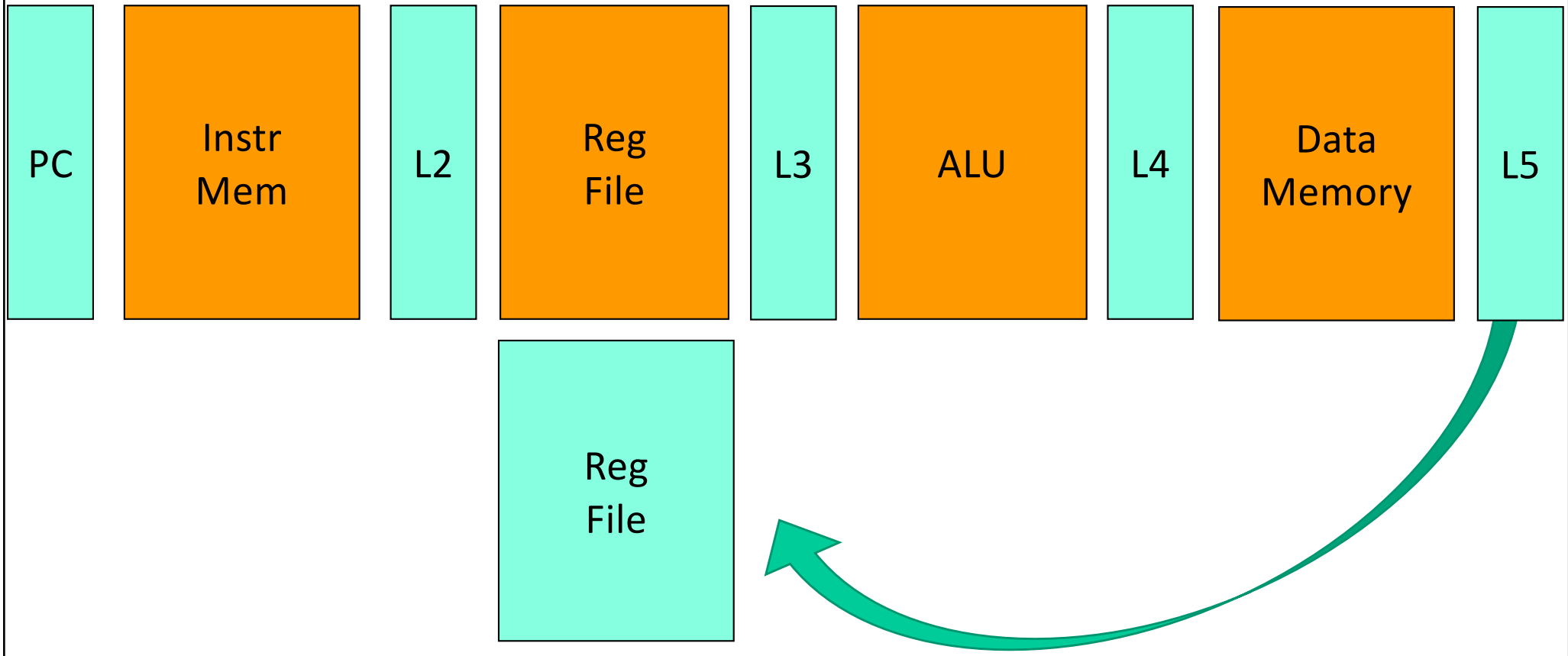
Latches and Clocks in a Single-Cycle Design



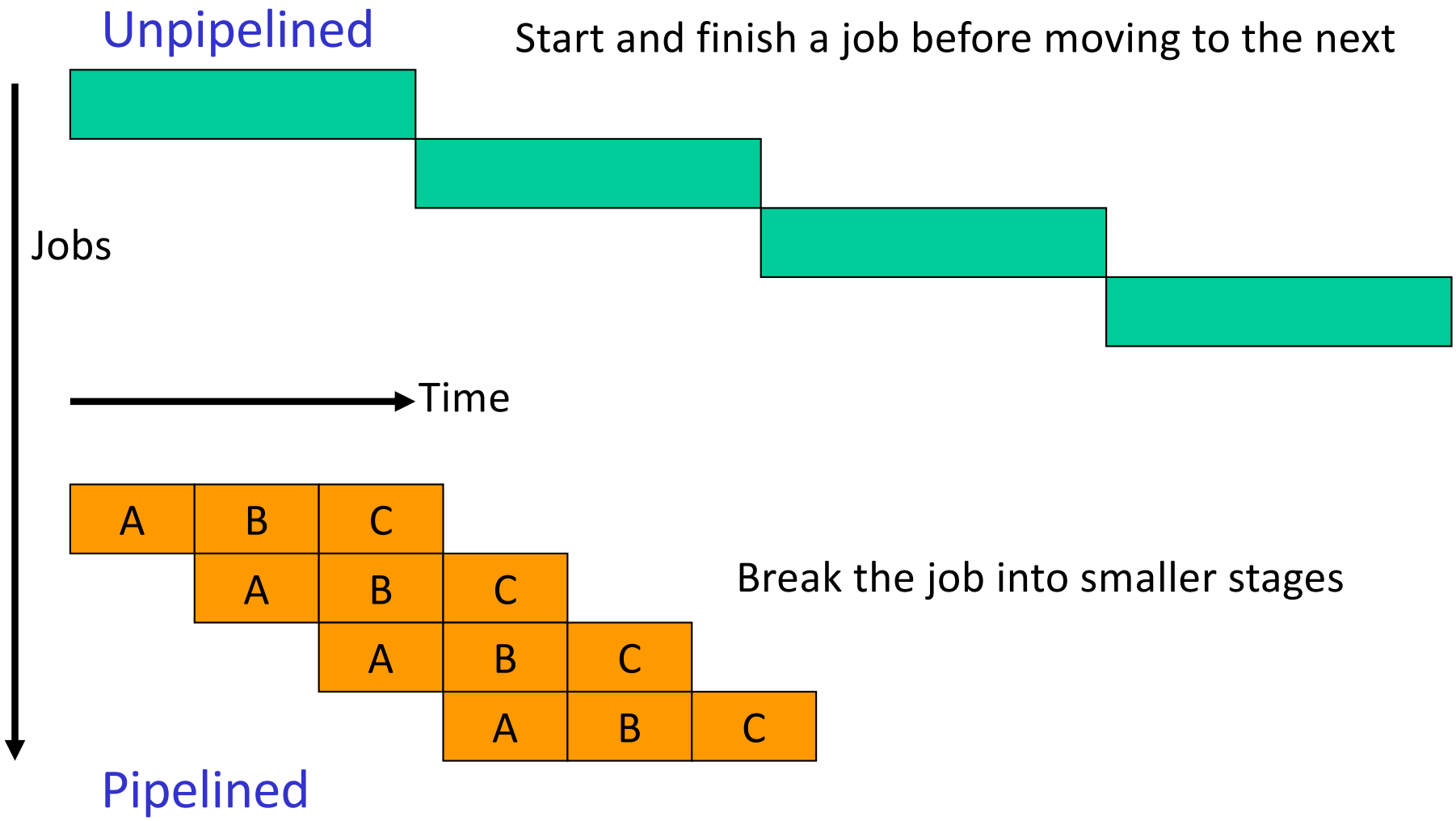
- The entire instruction executes in a single cycle
- Green blocks are latches
- At the rising edge, a new PC is recorded
- At the rising edge, the result of the previous cycle is recorded
- At the falling edge, the address of LW/SW is recorded so we can access the data memory in the 2nd half of the cycle

Multi-Stage Circuit

- Instead of executing the entire instruction in a single cycle (a single stage), let's break up the execution into multiple stages, each separated by a latch



The Assembly Line



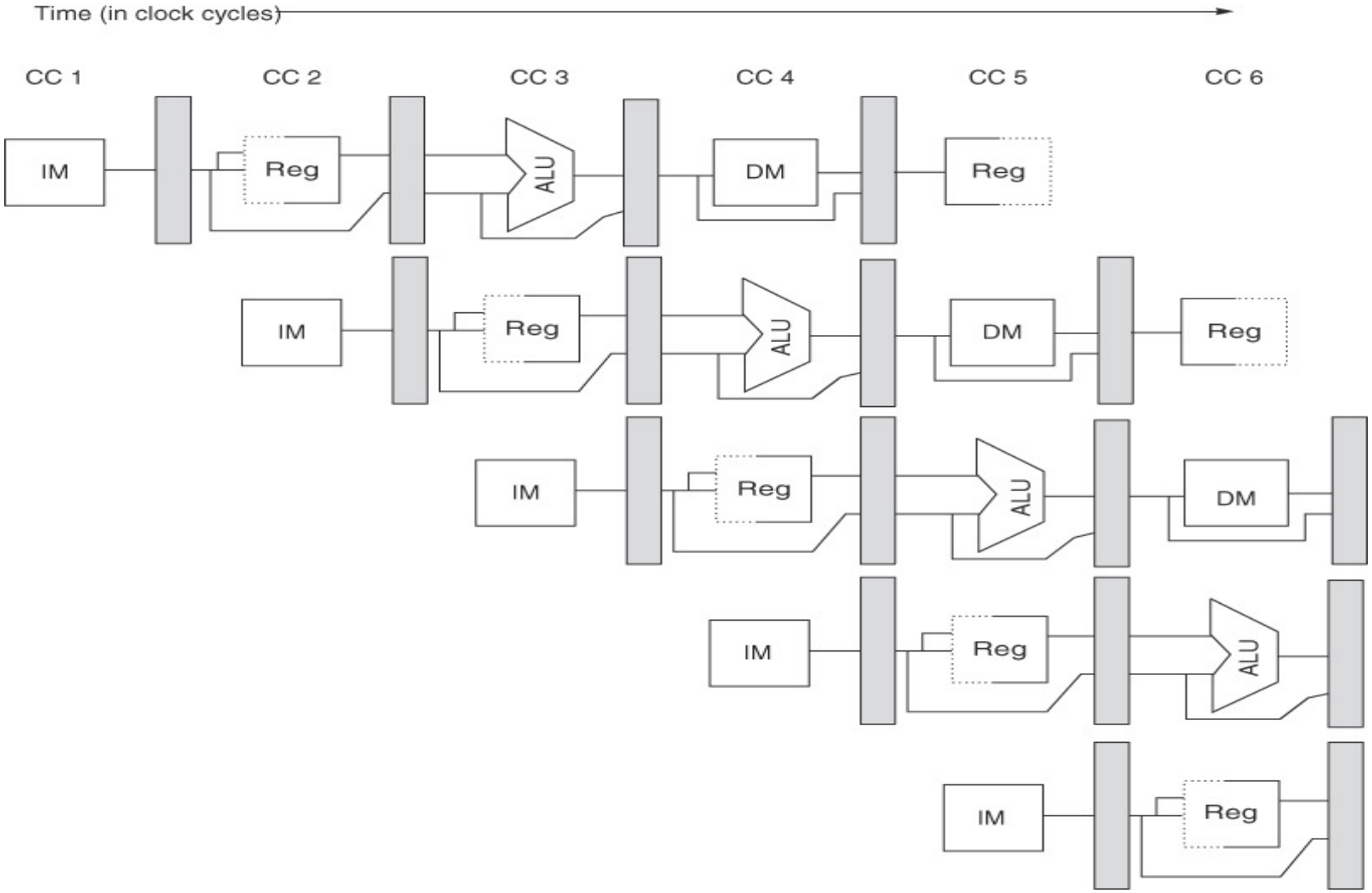
Performance Improvements?

- Does it take longer to finish each individual job?
- Does it take shorter to finish a series of jobs?
- What assumptions were made while answering these questions?
- Is a 10-stage pipeline better than a 5-stage pipeline?

Quantitative Effects

- As a result of pipelining:
  - Time in ns per instruction goes up
  - Each instruction takes more cycles to execute
  - But… average CPI remains roughly the same
  - Clock speed goes up
  - Total execution time goes down, resulting in lower average time per instruction
  - Under ideal conditions, speedup
    = ratio of *elapsed times between successive instruction completions*
    = number of pipeline stages = increase in clock speed

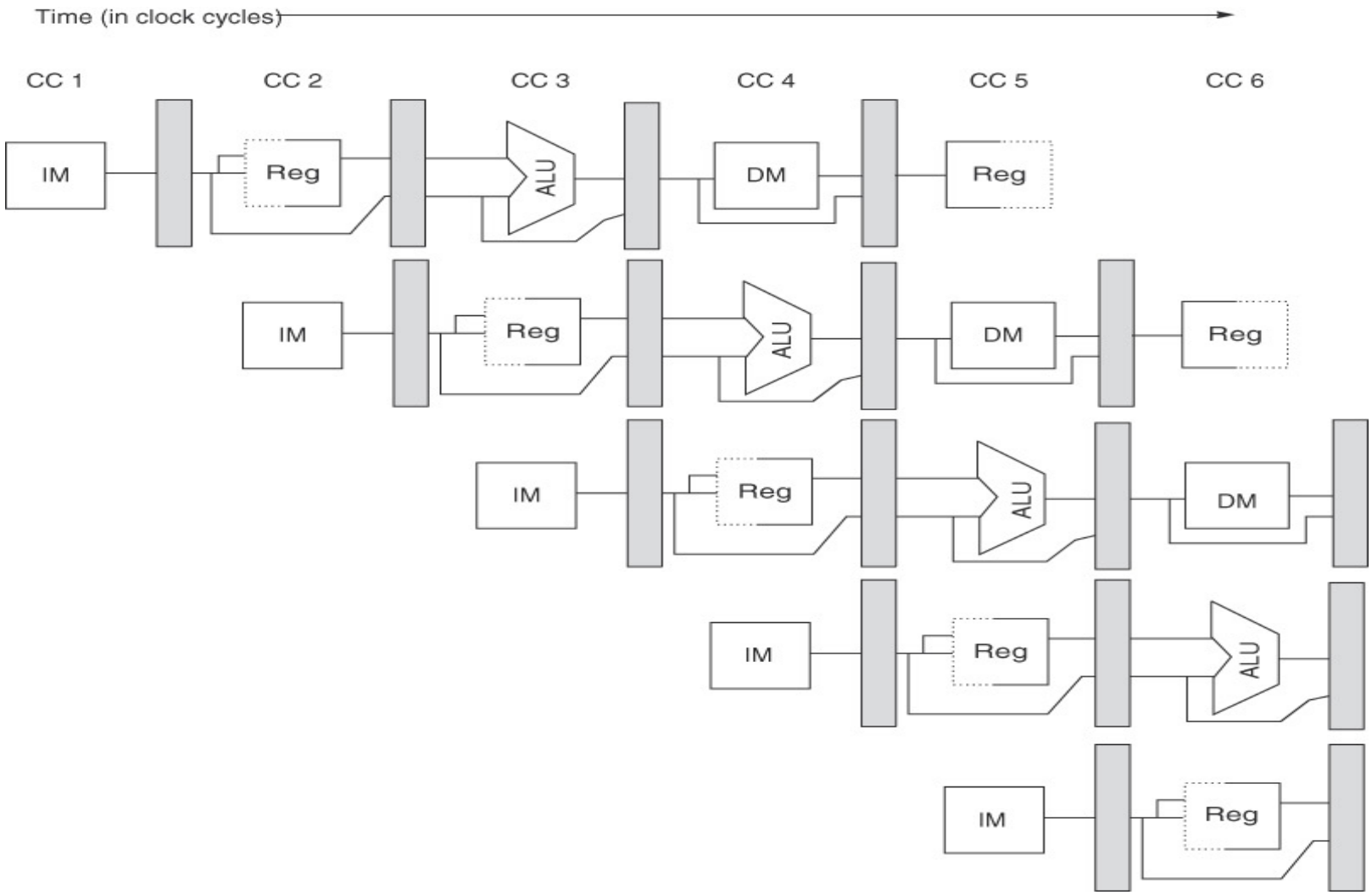
A 5-Stage Pipeline



Source: H&P textbook

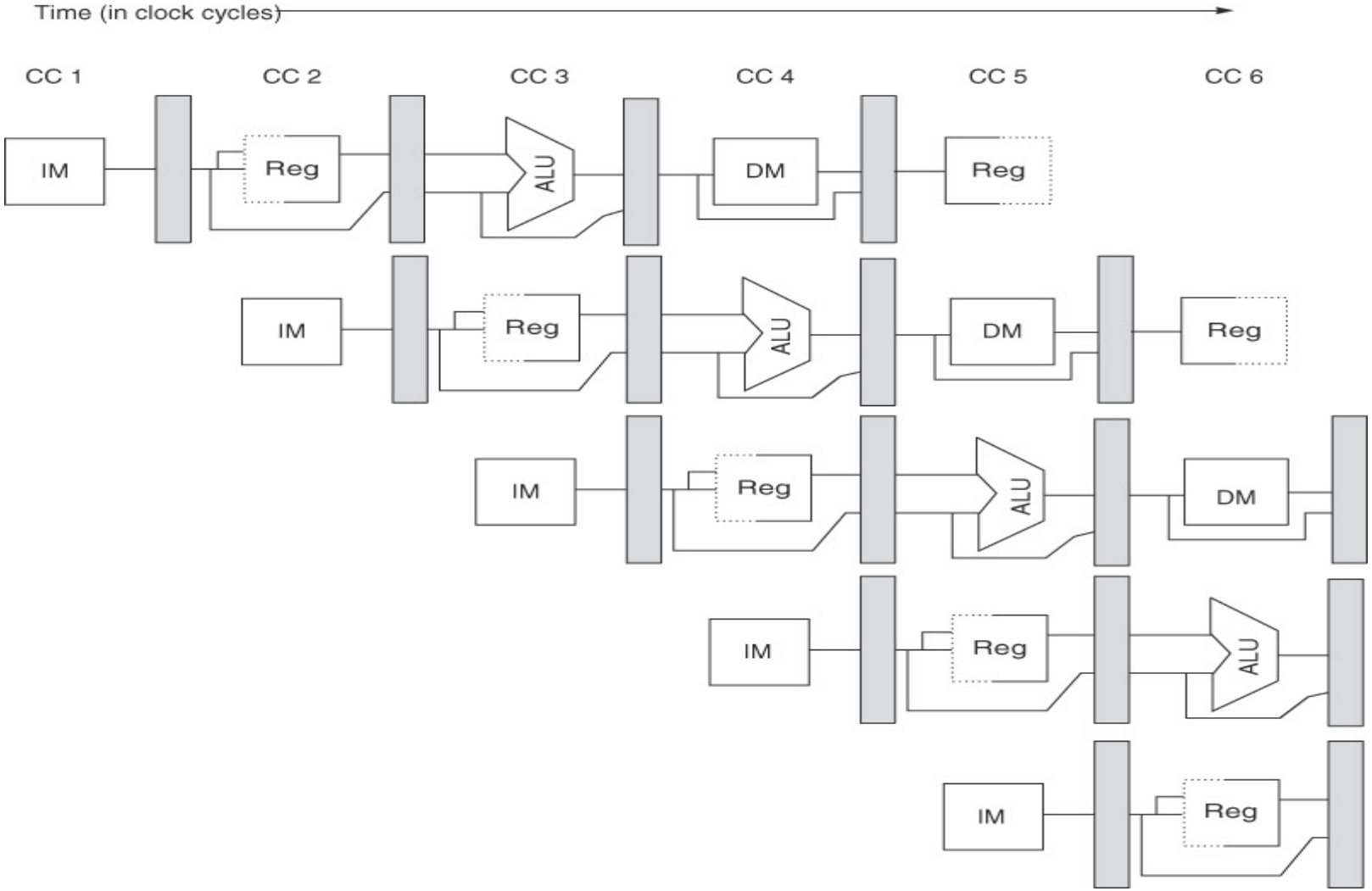
A 5-Stage Pipeline

Use the PC to access the I-cache and increment PC by 4



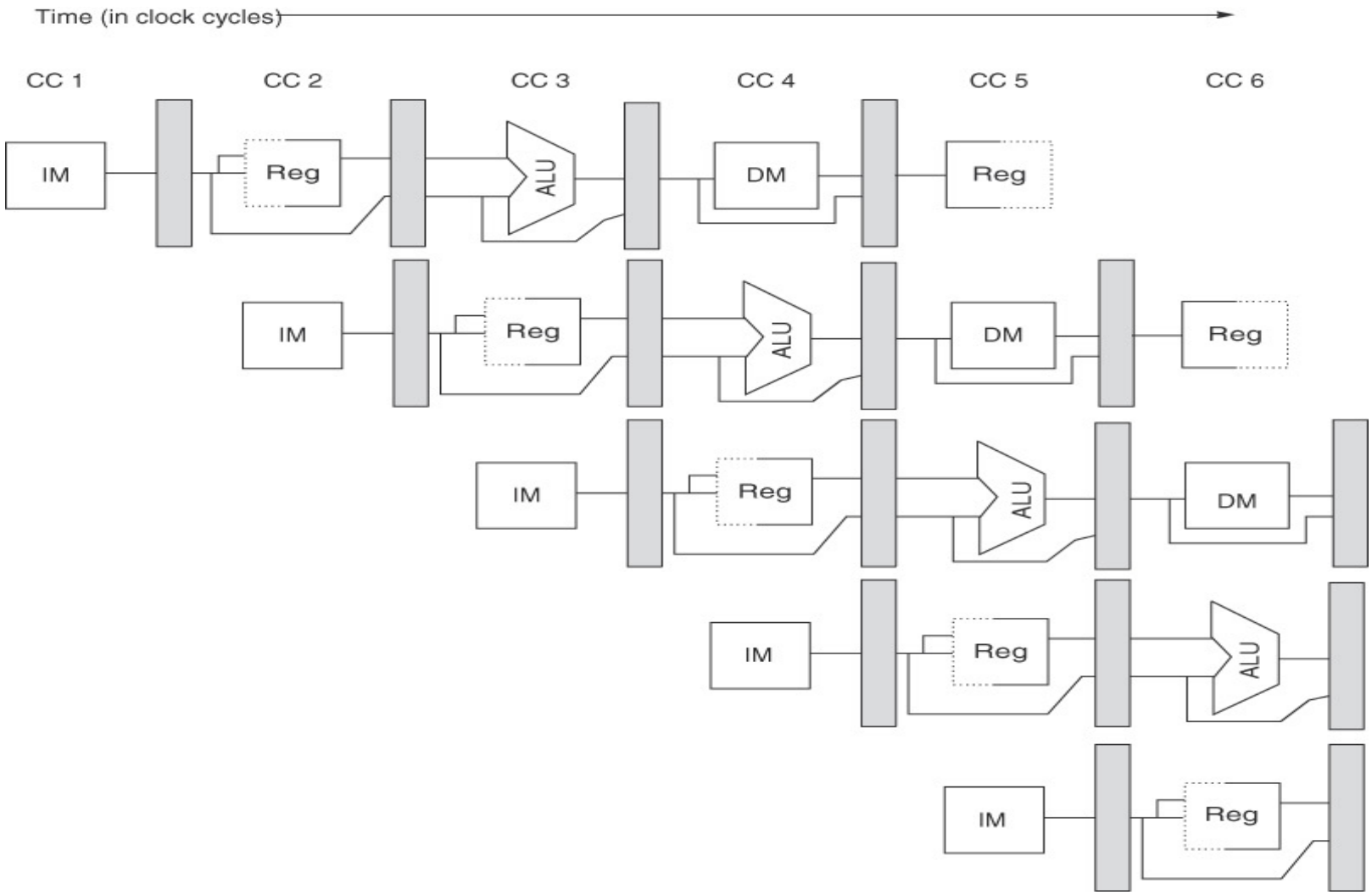
A 5-Stage Pipeline

Read registers, compare registers, compute branch target; for now, assume branches take 2 cyc (there is enough work that branches can easily take more)



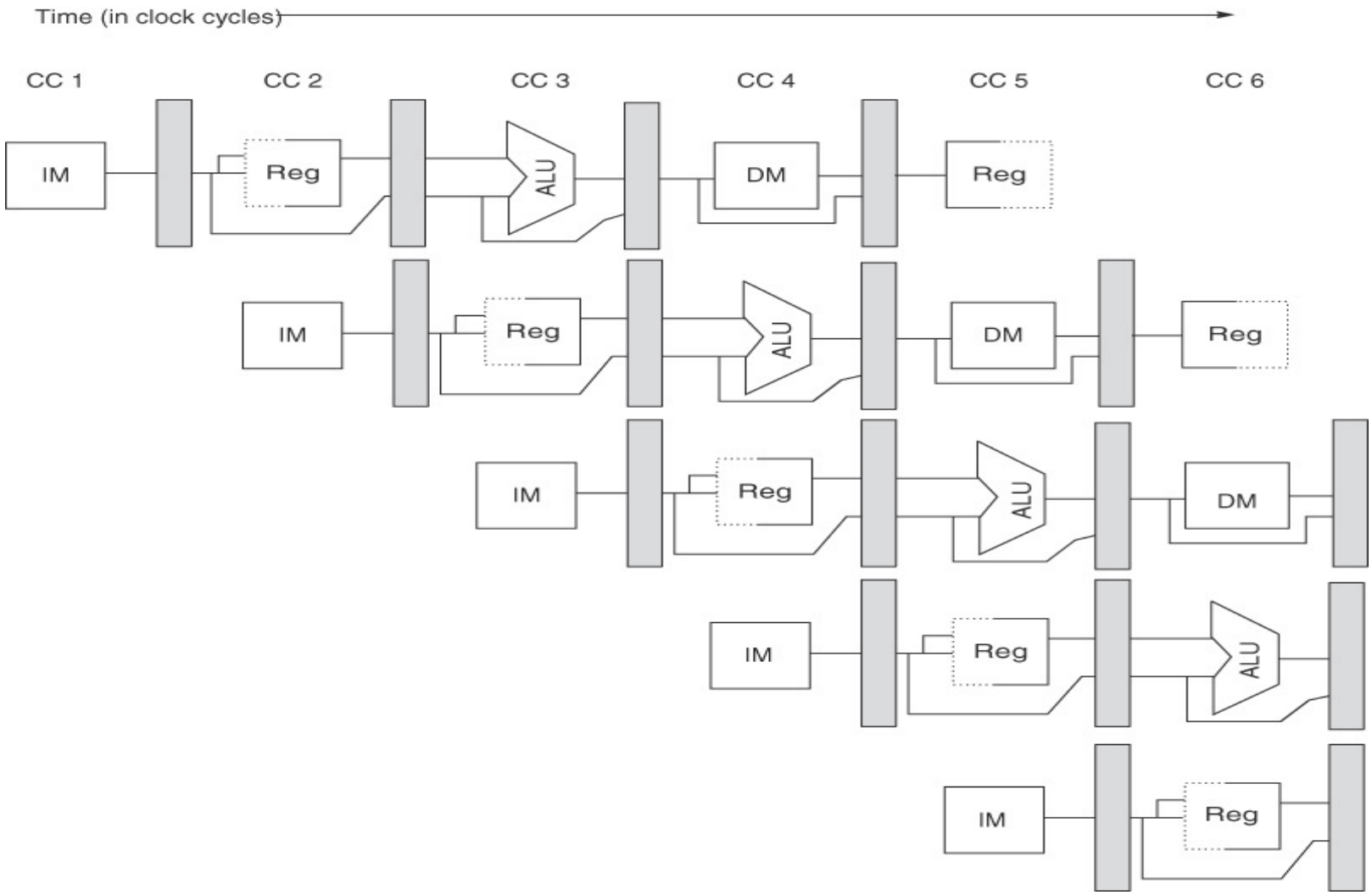
A 5-Stage Pipeline

ALU computation, effective address computation for load/store



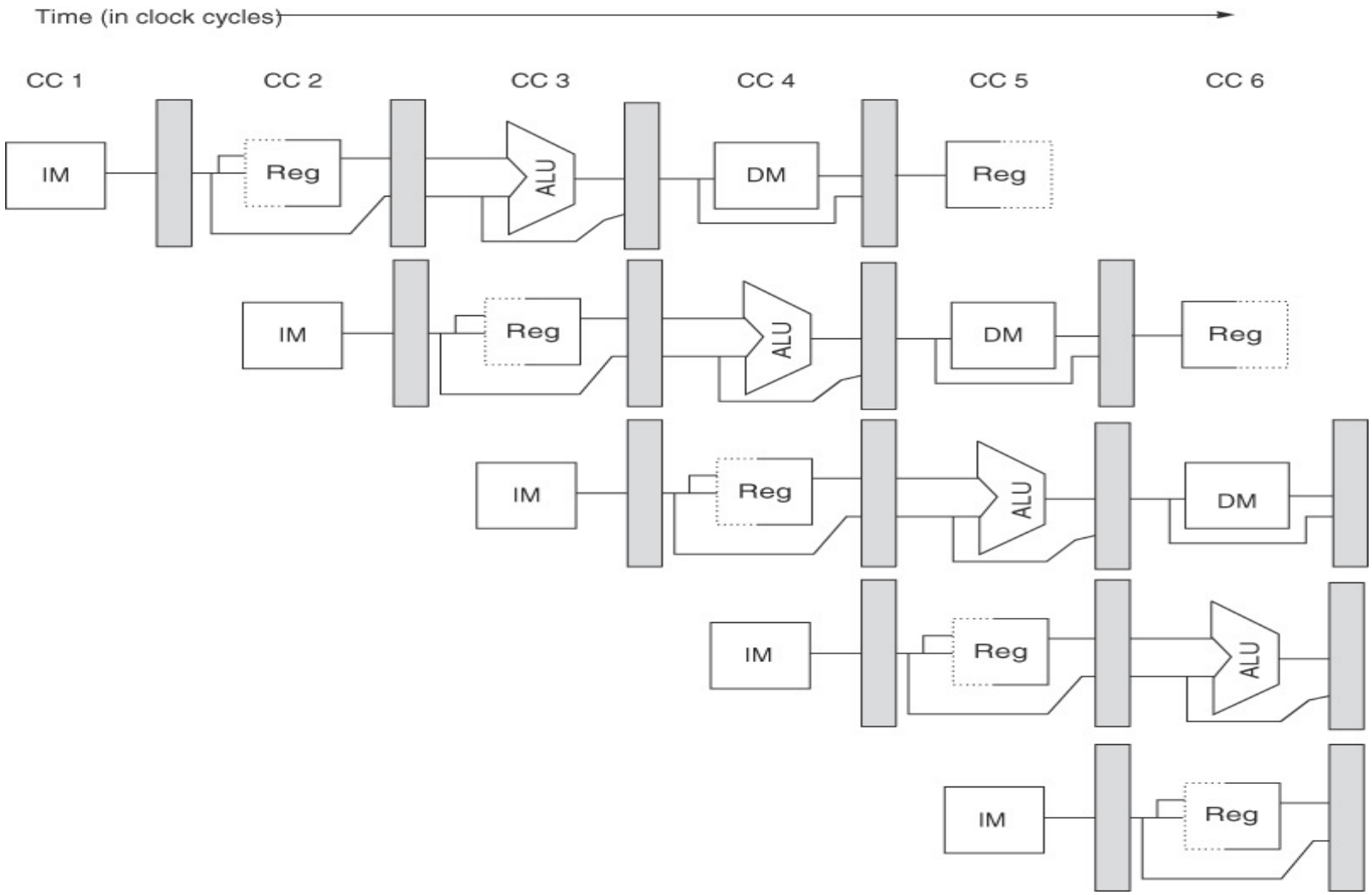
A 5-Stage Pipeline

Memory access to/from data cache, stores finish in 4 cycles



A 5-Stage Pipeline

Write result of ALU computation or load into register file



Pipeline Summary

	RR	ALU	DM	RW
ADD R1, R2, → R3	Rd R1,R2	R1+R2	--	Wr R3
BEQ R1, R2, 100	Rd R1, R2 Compare, Set PC	--	--	--
LD 8[R3] → R6	Rd R3	R3+8	Get data	Wr R6
ST 8[R3] ← R6	Rd R3,R6	R3+8	Wr data	--

Conflicts/Problems

- I-cache and D-cache are accessed in the same cycle – it helps to implement them separately
- Registers are read and written in the same cycle – easy to deal with if register read/write time equals cycle time/2
- Branch target changes only at the end of the second stage -- what do you do in the meantime?

Hazards

- Structural hazards: different instructions in different stages (or the same stage) conflicting for the same resource
- Data hazards: an instruction cannot continue because it needs a value that has not yet been generated by an earlier instruction
- Control hazard: fetch cannot continue because it does not know the outcome of an earlier branch – special case of a data hazard – separate category because they are treated in different ways